

Homework 03

Question 1: Build an explain-to-me skill

Answer

Completed per the assignment steps: designed `explain-to-me/SKILL.md`, installed it, and tested it by asking the agent to use `explain-to-me` on Question 2. The full contents of the developed skill file are appended at the end of this document under `explain-to-me skill`.

Question 2: Ridge regression with nearly collinear predictors

Answer

Part a

```
import numpy as np

np.random.seed(43231)
n = 100
L = np.random.randn(n)
U1 = np.random.randn(n)
U2 = np.random.randn(n)
X3 = np.random.randn(n)
X4 = np.random.randn(n)

# build near-collinear pair
X1 = L + 0.04 * U1
X2 = L + 0.04 * U2

X = np.column_stack([X1, X2, X3, X4])

# center and standardize: mean 0, average squared value 1
Xc = X - X.mean(axis=0, keepdims=True)
scale = np.sqrt((Xc**2).mean(axis=0))
X = Xc / scale # this is the standardized design matrix used throughout

# correlation between first two columns
corr12 = np.corrcoef(X[:, 0], X[:, 1])[0, 1]

# singular values via SVD
U, S, Vt = np.linalg.svd(X, full_matrices=False)

print("corr(X1, X2) =", corr12)
print("singular values:", S)
print("largest singular value:", S.max())
print("smallest singular value:", S.min())
```

```
corr(X1, X2) = 0.998268878796455
singular values: [14.15123473 10.1830787  9.7916542  0.41348737]
largest singular value: 14.15123472889276
smallest singular value: 0.41348736987744283
```

$\text{corr}(X_1, X_2) \approx 0.9983$, and the singular values are approximately $[14.151, 10.183, 9.792, 0.413]$, so $d_{\max} \approx 14.151$ and $d_{\min} \approx 0.413$.

Because X_1 and X_2 are both built from the same L_i plus a tiny independent perturbation, they are almost identical ($r \approx 0.998$), which makes $\mathbf{X}$ nearly rank-deficient. This shows up in the singular values: three are moderate-to-large ($\approx 14.15, 10.18, 9.79$), but the smallest is tiny ($d_{\min} \approx 0.41$) — that small value is the signature of the near-collinearity between X_1 and X_2 .

Since $\text{Var}(\hat{\beta}) = \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1} = \sigma^2 \mathbf{V} \mathbf{D}^{-2} \mathbf{V}^\top$, the variance of any contrast $\mathbf{c}^\top \hat{\beta}$ depends on how well $\mathbf{c}$ aligns with each singular direction, weighted by $1/d_j^2$. The contrast $\hat{\beta}_1 - \hat{\beta}_2$ essentially points along the near-null direction of $\mathbf{X}$ (since $X_1 - X_2 \approx 0.04(U_1 - U_2)$ is almost pure noise, carrying almost no signal), so its variance is inflated by roughly $1/d_{\min}^2$. The contrast $\hat{\beta}_1 + \hat{\beta}_2$, on the other hand, points along the direction of $2L_i$, which aligns with one of the large singular values, so its variance only scales like $1/d_{\max}^2$.

The ratio of these two variance-inflation factors is on the order of $(d_{\max}/d_{\min})^2 \approx (14.151/0.413)^2 \approx 1171$. So $\hat{\beta}_1 - \hat{\beta}_2$ should be dramatically more variable than $\hat{\beta}_1 + \hat{\beta}_2$: the data strongly pin down the total effect of the shared factor L_i (the sum), but there's almost no information in $\mathbf{X}$ to distinguish how much of that effect belongs to X_1 versus X_2 individually, so the split between $\hat{\beta}_1$ and $\hat{\beta}_2$ (their difference) is highly sensitive to noise in $\mathbf{y}$.

Part b

```
np.random.seed(43231)
n = 100

# --- generate X once (same as part a) ---
L = np.random.randn(n)
U1 = np.random.randn(n)
U2 = np.random.randn(n)
X3 = np.random.randn(n)
X4 = np.random.randn(n)

X1 = L + 0.04 * U1
X2 = L + 0.04 * U2

X = np.column_stack([X1, X2, X3, X4])
Xc = X - X.mean(axis=0, keepdims=True)
scale = np.sqrt((Xc**2).mean(axis=0))
X = Xc / scale
n, p = X.shape

beta_true = np.array([1.5, 1.5, 1.0, 0.0])
mu = X @ beta_true

lambdas = [0, 0.02, 0.2]
nrep = 200

results = {lam: {"b1": [], "b2": [], "sum": [], "diff": [],
                 "trainMSE": [], "testMSE": []} for lam in lambdas}

for rep in range(nrep):
    eps = np.random.randn(n)
    eps_star = np.random.randn(n)
    y = mu + eps
    ystar = mu + eps_star

    ybar = y.mean()
    ytilde = y - ybar

    for lam in lambdas:
        A = X.T @ X + n * lam * np.eye(p)
        b = np.linalg.solve(A, X.T @ ytilde)

        beta0 = ybar
        fitted = beta0 + X @ b          # same X used for train and test

        train_mse = np.mean((y - fitted) ** 2)
        test_mse = np.mean((ystar - fitted) ** 2)

        results[lam]["b1"].append(b[0])
        results[lam]["b2"].append(b[1])
        results[lam]["sum"].append(b[0] + b[1])
        results[lam]["diff"].append(b[0] - b[1])
        results[lam]["trainMSE"].append(train_mse)
        results[lam]["testMSE"].append(test_mse)

for lam in lambdas:
    print(f"\nlambda = {lam}")
    for key in ["b1", "b2", "sum", "diff"]:
        arr = np.array(results[lam][key])
        print(f"  {key:5s}: mean = {arr.mean():.7f}, sd = {arr.std(ddof=1):.7f}")
    print(f"  train MSE mean: {np.mean(results[lam]['trainMSE']):.4f}")
    print(f"  test MSE mean: {np.mean(results[lam]['testMSE']):.4f}")

lambda = 0
b1 : mean = 1.4482, sd = 1.6289
b2 : mean = 1.5505, sd = 1.6295
sum : mean = 2.9987, sd = 0.0919
```

```
diff : mean = -0.1023, sd = 3.2570
train MSE mean: 0.9332
test MSE mean: 1.0475
```

```
lambda = 0.02
b1 : mean = 1.4827, sd = 0.1357
b2 : mean = 1.4871, sd = 0.1364
sum : mean = 2.9698, sd = 0.0910
diff : mean = -0.0044, sd = 0.2565
train MSE mean: 0.9421
test MSE mean: 1.0402
```

```
lambda = 0.2
b1 : mean = 1.3674, sd = 0.0438
b2 : mean = 1.3649, sd = 0.0441
sum : mean = 2.7323, sd = 0.0835
diff : mean = 0.0025, sd = 0.0276
train MSE mean: 1.0420
test MSE mean: 1.1328
```

λ	$\hat{\beta}_1$ mean (sd)	$\hat{\beta}_2$ mean (sd)	$\hat{\beta}_1 + \hat{\beta}_2$ mean (sd)	$\hat{\beta}_1 - \hat{\beta}_2$ mean (sd)	Train MSE	Test MSE
0 (OLS)	1.448 (1.629)	1.551 (1.630)	2.999 (0.092)	-0.102 (3.257)	0.933	1.048
0.02	1.483 (0.136)	1.487 (0.136)	2.970 (0.091)	-0.004 (0.257)	0.942	1.040
0.2	1.367 (0.044)	1.365 (0.044)	2.732 (0.084)	0.003 (0.028)	1.042	1.133

The simulation confirms the intuition from part (a). At $\lambda = 0$ (OLS), both $\hat{\beta}_1$ and $\hat{\beta}_2$ are essentially unbiased (means near 1.5) but extremely noisy, with standard deviations around 1.6 — a direct consequence of the tiny singular value associated with the near-collinear pair. The sum $\hat{\beta}_1 + \hat{\beta}_2$ is already tightly concentrated (sd ≈ 0.09) even without penalty, since it lives in the well-estimated large-singular-value direction, while the difference $\hat{\beta}_1 - \hat{\beta}_2$ is wildly unstable (sd ≈ 3.26), dwarfing the true value of zero, since it lives in the near-null direction.

As λ increases from 0 to 0.02 to 0.2, the individual coefficients and especially the difference contract sharply toward more stable values: the sd of $\hat{\beta}_1 - \hat{\beta}_2$ falls from 3.26 to 0.26 to 0.03, while the sd of the sum barely moves (0.092 $\rightarrow$ 0.091 $\rightarrow$ 0.084). This asymmetric shrinkage is exactly what ridge regression targets — it suppresses variance most in directions with small singular values.

Test MSE is minimized at the small penalty $\lambda = 0.02$ (1.040), slightly beating OLS (1.048), reflecting the classic bias–variance tradeoff: a small amount of shrinkage removes a large amount of variance in the unstable difference direction at almost no bias cost. At $\lambda = 0.2$ the penalty is too strong — it begins biasing the well-estimated sum direction as well, and test MSE rises to 1.133. Training MSE increases monotonically with λ (0.933 $\rightarrow$ 0.942 $\rightarrow$ 1.042), as expected, since ridge always fits the training data somewhat worse by construction, but this does not track test performance in the same way.

Part c

Individual coefficients. At $\lambda = 0$ (OLS), $\hat{\beta}_1$ and $\hat{\beta}_2$ are essentially unbiased — their means (1.448, 1.551) are close to the true values (1.5, 1.5) — but their variance is enormous (sd ≈ 1.6 each), because the near-collinearity between X_1 and X_2 leaves the data almost unable to tell OLS how to split credit for the shared signal between the two predictors. As λ increases to 0.02 and then 0.2, the standard deviations collapse (to about 0.14 and then 0.04), but at the cost of growing bias: the means drift downward (to about 1.48, then 1.37) as ridge shrinks coefficients toward zero. This is the standard ridge bias–variance tradeoff, but it plays out very unevenly across directions.

Sum vs. difference. $\hat{\beta}_1 + \hat{\beta}_2$ is already low-variance and nearly unbiased even at $\lambda = 0$ (sd ≈ 0.09), because it lies along the well-determined, large-singular-value direction of $\mathbf{X}$ (essentially the shared factor L_i); ridge shrinks its mean only modestly as λ grows and barely touches its variance. In sharp contrast, $\hat{\beta}_1 - \hat{\beta}_2$ is wildly unstable at $\lambda = 0$ (sd ≈ 3.26 , far exceeding the entire signal) because it is driven almost entirely by noise in the near-null singular direction. Even a tiny penalty ($\lambda = 0.02$) collapses its sd to 0.26, and $\lambda = 0.2$ nearly eliminates the variability (sd ≈ 0.03), correctly reflecting that the true contrast $\beta_1 - \beta_2 = 0$.

Test MSE and the bias–variance tradeoff. Test MSE is minimized at the small penalty $\lambda = 0.02$ (1.040), just below OLS’s 1.048, and worse at $\lambda = 0.2$ (1.133). This is a textbook bias–variance picture: a small ridge penalty removes a huge amount of variance in the unstable near-null direction at essentially no bias cost, improving prediction slightly over OLS. Push the penalty too far (0.2), however, and it starts biasing the well-estimated signal-carrying direction as well — now the added bias outweighs the (already small) further variance reduction, and test MSE rises. Training MSE, by contrast, increases monotonically with λ (0.933 $\rightarrow$ 0.942 $\rightarrow$ 1.042), since ridge always fits the training data somewhat worse by construction — but training MSE is not a reliable guide to test performance, which is exactly the point of this exercise.

Why ridge stabilizes coefficients without changing fitted values as much. In the SVD basis $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, ridge regression shrinks the coefficient along each principal direction j by the factor

$$\frac{d_j^2}{d_j^2 + n\lambda},$$

and the fitted values $\mathbf{X}\hat{\boldsymbol{\beta}}$ depend on these shrunk coefficients weighted by d_j in a way that keeps strong directions almost untouched. For a direction with a *large* singular value (like the shared $X_1 + X_2$ signal direction, which carries almost all of the true predictive information), $d_j^2 \gg n\lambda$ for the small λ ’s used here, so the shrinkage factor stays close to 1 — that part of the fit, and hence the fitted values and test predictions, changes very little. But for the direction with a *tiny* singular value (the $X_1 - X_2$ contrast, which is almost pure noise with essentially no real signal), the same $n\lambda$ term dominates d_j^2 , so the shrinkage factor collapses toward 0 quickly.

Because the individual coefficients $\hat{\beta}_1$ and $\hat{\beta}_2$ are linear combinations of *both* the strong (sum) and weak (difference) directions, suppressing the noisy weak direction alone produces a large, visible stabilization in $\hat{\beta}_1$ and $\hat{\beta}_2$ individually. But since that weak direction carried almost no real signal to begin with, removing its noisy contribution barely changes the fitted values or predictions — the meaningful part of the fit (driven by the strong, shared-signal direction) survives almost untouched. This is why ridge can dramatically stabilize individual, poorly-identified coefficients in the presence of collinearity while leaving overall predictive performance nearly unchanged (or, as seen here with $\lambda = 0.02$, even slightly improved).

Question 3: What does ridge regression shrink?

Answer

Part a

Derivation. The ridge objective is

$$L_\lambda(\beta) = \frac{1}{2n} \|\tilde{\mathbf{y}} - \mathbf{X}\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2.$$

Differentiate with respect to β and set the gradient to zero:

$$\nabla_\beta L_\lambda(\beta) = -\frac{1}{n} \mathbf{X}^\top (\tilde{\mathbf{y}} - \mathbf{X}\beta) + \lambda\beta = \mathbf{0}.$$

Multiplying through by n :

$$-\mathbf{X}^\top \tilde{\mathbf{y}} + \mathbf{X}^\top \mathbf{X}\beta + n\lambda\beta = \mathbf{0} \implies (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p) \hat{\beta}_\lambda = \mathbf{X}^\top \tilde{\mathbf{y}}.$$

Since L_λ is a quadratic function with Hessian $\frac{1}{n} \mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_p \succeq \lambda \mathbf{I}_p \succ 0$ (for $\lambda > 0$), this is not just a stationary point but the unique global minimizer.

Why the solution is unique for any $\lambda > 0$. $\mathbf{X}^\top \mathbf{X}$ is symmetric positive semi-definite, so all its eigenvalues satisfy $\mu_k \geq 0$ — this holds regardless of whether $\mathbf{X}$ is rank-deficient or $p > n$ (in which case $\mathbf{X}^\top \mathbf{X}$ is necessarily singular, with some $\mu_k = 0$). Adding $n\lambda \mathbf{I}_p$ shifts every eigenvalue by $n\lambda > 0$, so the eigenvalues of $\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p$ are $\mu_k + n\lambda \geq n\lambda > 0$ for all k . A symmetric matrix with strictly positive eigenvalues is positive definite, hence invertible. Therefore $\hat{\beta}_\lambda = (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p)^{-1} \mathbf{X}^\top \tilde{\mathbf{y}}$ exists and is unique — even when $\mathbf{X}^\top \mathbf{X}$ itself is singular (rank-deficient design, or $p > n$), the ridge penalty regularizes exactly the directions where $\mathbf{X}^\top \mathbf{X}$ has zero (or near-zero) eigenvalues.

Part b

Setup. Let $\mathbf{X} = \mathbf{U}_r \mathbf{D}_r \mathbf{V}_r^\top$ be the compact SVD, with $\mathbf{U}_r^\top \mathbf{U}_r = \mathbf{I}_r$, $\mathbf{V}_r^\top \mathbf{V}_r = \mathbf{I}_r$, and $\mathbf{D}_r = \text{diag}(d_1, \dots, d_r)$, $d_j > 0$.

Deriving $\hat{\beta}_\lambda$. Extend $\mathbf{V}_r$ to a full orthogonal basis $\mathbf{V} = [\mathbf{V}_r \ \mathbf{V}_r^\perp] \in \mathbb{R}^{p \times p}$. Since $\mathbf{X}^\top \mathbf{X} = \mathbf{V}_r \mathbf{D}_r^2 \mathbf{V}_r^\top$ and $\mathbf{I}_p = \mathbf{V}_r \mathbf{V}_r^\top + \mathbf{V}_r^\perp \mathbf{V}_r^{\perp\top}$,

$$\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p = \mathbf{V}_r (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r) \mathbf{V}_r^\top + n\lambda \mathbf{V}_r^\perp \mathbf{V}_r^{\perp\top}.$$

Because $\mathbf{V}_r$ and $\mathbf{V}_r^\perp$ are orthogonal blocks, this inverts blockwise:

$$(\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p)^{-1} = \mathbf{V}_r (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r)^{-1} \mathbf{V}_r^\top + \frac{1}{n\lambda} \mathbf{V}_r^\perp \mathbf{V}_r^{\perp\top}.$$

Multiplying by $\mathbf{X}^\top \tilde{\mathbf{y}} = \mathbf{V}_r \mathbf{D}_r \mathbf{U}_r^\top \tilde{\mathbf{y}}$, the second term vanishes (since $\mathbf{V}_r^{\perp\top} \mathbf{V}_r = \mathbf{0}$), leaving

$$\hat{\beta}_\lambda = \mathbf{V}_r (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r)^{-1} \mathbf{D}_r \mathbf{U}_r^\top \tilde{\mathbf{y}}.$$

Fitted values.

$$\mathbf{X} \hat{\beta}_\lambda = \mathbf{U}_r \mathbf{D}_r \mathbf{V}_r^\top \hat{\beta}_\lambda = \mathbf{U}_r \mathbf{D}_r (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r)^{-1} \mathbf{D}_r \mathbf{U}_r^\top \tilde{\mathbf{y}} = \mathbf{U}_r \text{diag}\left(\frac{d_j^2}{d_j^2 + n\lambda}\right) \mathbf{U}_r^\top \tilde{\mathbf{y}}.$$

Writing $\tilde{\mathbf{y}} = \sum_{j=1}^r (\mathbf{u}_j^\top \tilde{\mathbf{y}}) \mathbf{u}_j + (\text{component orthogonal to } \mathbf{U}_r)$, the fitted value is

$$\mathbf{X} \hat{\beta}_\lambda = \sum_{j=1}^r \rho_j(\lambda) (\mathbf{u}_j^\top \tilde{\mathbf{y}}) \mathbf{u}_j, \quad \rho_j(\lambda) = \frac{d_j^2}{d_j^2 + n\lambda}.$$

So the component of $\tilde{\mathbf{y}}$ along each left singular direction $\mathbf{u}_j$ is scaled by exactly $\rho_j(\lambda) \in (0, 1)$ — this **is** the shrinkage factor. (For comparison, OLS, $\lambda = 0$, gives $\rho_j = 1$ for every j with $d_j > 0$: no shrinkage.)

Numerical example. With $n = 100$, $\lambda = 0.01 \Rightarrow n\lambda = 1$:

```
n = 100
lam = 0.01
n_lam = n * lam # = 1

d = [10, 2, 0.2]
rho = [d_j**2 / (d_j**2 + n_lam) for d_j in d]

for j, (dj, rj) in enumerate(zip(d, rho), start=1):
    print(f"d_{j} = {dj:>5}: rho_{j}(lambda) = {rj:.4f}")
```

```
df_eff = 1 + sum(rho)
print(f"\nEffective df (including intercept) = {df_eff:.4f}")
```

```
d_1 = 10: rho_1(lambda) = 0.9901
d_2 = 2: rho_2(lambda) = 0.8000
d_3 = 0.2: rho_3(lambda) = 0.0385
```

Effective df (including intercept) = 2.8286

$$\rho_1(\lambda) = \frac{100}{101} \approx 0.9901, \quad \rho_2(\lambda) = \frac{4}{5} = 0.8000, \quad \rho_3(\lambda) = \frac{0.04}{1.04} \approx 0.0385,$$

$$\text{df}_{\text{eff}}(\lambda) = 1 + 0.9901 + 0.8000 + 0.0385 \approx 2.8286.$$

Part c

Strongest shrinkage. The third direction ($d_3 = 0.2$) receives by far the strongest shrinkage, $\rho_3 \approx 0.038$ — its contribution to the fitted values is reduced to under 4% of what OLS would use. In general $\rho_j(\lambda)$ is increasing in d_j : large singular values (strong, well-determined directions of $\mathbf{X}$) are barely shrunk, while small singular values (weak, poorly-determined directions) are shrunk almost to zero.

Why shrinking this direction reduces prediction variance. For OLS, the variance of the fitted-value component along $\mathbf{u}_j$ is σ^2 times a weight that grows like $1/d_j^2$ once you propagate back through $\hat{\beta}$ — equivalently, the corresponding coefficient-space direction $\mathbf{v}_j$ has $\text{Var}(\mathbf{v}_j^\top \hat{\beta}_{\text{OLS}}) = \sigma^2/d_j^2$, which is huge when d_j is small. Ridge's shrinkage factor $\rho_j(\lambda) = d_j^2/(d_j^2 + n\lambda)$ multiplies that variance by $\rho_j(\lambda)^2$, so the ridge-estimated variance in that direction is $\sigma^2 d_j^2/(d_j^2 + n\lambda)^2$, which is dramatically smaller than the OLS variance whenever $n\lambda \gg d_j^2$ (as with $d_3 = 0.2$ here). Because this direction contributes little signal to $\mathbf{X}$ to begin with (small d_j means the predictors vary little along $\mathbf{v}_j$), most of what OLS is fitting there is noise — so suppressing it removes mostly variance, not information.

When this same shrinkage could produce substantial bias. If the *true* coefficient vector β^* has a large projection onto $\mathbf{v}_3$ — i.e., the true signal genuinely depends strongly on the weak (small- d_3) direction of $\mathbf{X}$ — then aggressively shrinking that direction discards real signal, not just noise. In that case $\mathbb{E}[\mathbf{v}_3^\top \hat{\beta}_\lambda] = \rho_3(\lambda) \mathbf{v}_3^\top \beta^*$ is pulled far from the true value $\mathbf{v}_3^\top \beta^*$, producing substantial bias in both the coefficients and the fitted values along that direction. This is the core tension: ridge shrinkage helps most when weak singular directions carry mostly noise (as engineered in Question 2 by nearly collinear predictors), but hurts when weak directions happen to carry real signal — the practitioner cannot tell which is the case from d_j alone, which is why λ must be tuned (e.g., by cross-validation) to balance this bias-variance tradeoff rather than fixed a priori.

Question 4: Ridge regression and optimization

Answer

Part a

The condition number of a symmetric positive-definite matrix is the ratio of its largest to smallest eigenvalue.

```
eig_A0 = [9, 0.01]
lam = 0.09
eig_Alam = [e + lam for e in eig_A0]

cond_A0 = max(eig_A0) / min(eig_A0)
cond_Alam = max(eig_Alam) / min(eig_Alam)

print("Eigenvalues of A0:", eig_A0, " -> condition number:", cond_A0)
print("Eigenvalues of A_lambda:", eig_Alam, " -> condition number:", cond_Alam)
```

```
Eigenvalues of A0: [9, 0.01] -> condition number: 900.0
Eigenvalues of A_lambda: [9.09, 0.09999999999999999] -> condition number: 90.9
```

$$\kappa(\mathbf{A}_0) = \frac{9}{0.01} = 900, \quad \kappa(\mathbf{A}_\lambda) = \frac{9.09}{0.10} = 90.9.$$

Interpretation. Since $\mathbf{A}_\lambda = \mathbf{A}_0 + \lambda \mathbf{I}_p$ shares $\mathbf{A}_0$'s eigenvectors and simply shifts every eigenvalue up by λ , the ridge penalty raises the *smallest* eigenvalue proportionally far more than the largest one (in relative terms: $0.01 \rightarrow 0.10$ is a 10-fold increase, while $9 \rightarrow 9.09$ is barely a 1% increase). This flattens the eigenvalue spectrum and shrinks the condition number by a factor of 10, from 900 down to 90.9. Geometrically, the level sets of L_0 (unpenalized least squares) are extremely elongated ellipses — nearly flat in the direction of the small eigenvalue — while the level sets of L_λ are much closer to circular. A more circular (well-conditioned) objective is much friendlier to gradient-based optimization: gradient descent zig-zags badly on elongated, ill-conditioned bowls, but converges quickly and directly on well-conditioned ones.

Part b

Deriving the recursion. The ridge loss is $L_\lambda(\beta) = \frac{1}{2n} \|\tilde{\mathbf{y}} - \mathbf{X}\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2$, with gradient

$$\nabla L_\lambda(\beta) = \frac{1}{n} \mathbf{X}^\top \mathbf{X} \beta - \frac{1}{n} \mathbf{X}^\top \tilde{\mathbf{y}} + \lambda \beta = \mathbf{A}_\lambda \beta - \frac{1}{n} \mathbf{X}^\top \tilde{\mathbf{y}}.$$

At the minimizer, $\nabla L_\lambda(\hat{\beta}_\lambda) = \mathbf{0}$, so $\frac{1}{n} \mathbf{X}^\top \tilde{\mathbf{y}} = \mathbf{A}_\lambda \hat{\beta}_\lambda$. Substituting back,

$$\nabla L_\lambda(\beta) = \mathbf{A}_\lambda \beta - \mathbf{A}_\lambda \hat{\beta}_\lambda = \mathbf{A}_\lambda (\beta - \hat{\beta}_\lambda).$$

So the gradient descent update becomes

$$\beta^{(k+1)} - \hat{\beta}_\lambda = (\beta^{(k)} - \hat{\beta}_\lambda) - \eta \mathbf{A}_\lambda (\beta^{(k)} - \hat{\beta}_\lambda) \implies \mathbf{e}^{(k+1)} = (\mathbf{I}_p - \eta \mathbf{A}_\lambda) \mathbf{e}^{(k)}.$$

Since $\mathbf{A}_\lambda = \mathbf{A}_0 + \lambda \mathbf{I}_p$ is diagonalized by the same eigenvectors as $\mathbf{A}_0$, and an eigendirection with unpenalized eigenvalue a (eigenvalue of $\mathbf{A}_0$) has eigenvalue $a + \lambda$ under $\mathbf{A}_\lambda$, the corresponding eigenvalue of $\mathbf{I}_p - \eta \mathbf{A}_\lambda$ is $1 - \eta(a + \lambda)$. Hence the component of $\mathbf{e}^{(k)}$ along that eigendirection is multiplied by exactly

$$1 - \eta(a + \lambda)$$

at every iteration, as required.

Interval of step sizes that contracts both directions. The two eigenvalues of $\mathbf{A}_\lambda$ are $M = 9.09$ and $m = 0.10$. Contraction in a given direction requires $|1 - \eta\mu| < 1$, i.e. $0 < \eta < 2/\mu$. For *both* directions to contract simultaneously, η must satisfy the tighter (larger-eigenvalue) constraint:

$$0 < \eta < \frac{2}{M} = \frac{2}{9.09} \approx 0.220.$$

```
M = 9.09
m = 0.10

eta_bound = 2 / M
print(f"Contracts-everywhere interval: 0 < eta < {eta_bound:.4f}")

# eta = 1/M
eta1 = 1 / M
factor_M_1 = 1 - eta1 * M
factor_m_1 = 1 - eta1 * m
print(f"\neta = 1/M = {eta1:.4f}")
print(f"  contraction factor (M direction) = {factor_M_1:.4f}")
print(f"  contraction factor (m direction) = {factor_m_1:.4f}")

# eta = 2.1/M
eta2 = 2.1 / M
factor_M_2 = 1 - eta2 * M
factor_m_2 = 1 - eta2 * m
print(f"\neta = 2.1/M = {eta2:.4f}")
print(f"  factor (M direction) = {factor_M_2:.4f}  (|.| = {abs(factor_M_2):.4f})")
print(f"  factor (m direction) = {factor_m_2:.4f}  (|.| = {abs(factor_m_2):.4f})")
```

Contracts-everywhere interval: 0 < eta < 0.2200

```
eta = 1/M = 0.1100
  contraction factor (M direction) = 0.0000
  contraction factor (m direction) = 0.9890

eta = 2.1/M = 0.2310
  factor (M direction) = -1.1000  (|.| = 1.1000)
  factor (m direction) = 0.9769  (|.| = 0.9769)
```

$\eta = 1/M \approx 0.110$.

$$1 - \eta M = 1 - 1 = 0, \quad 1 - \eta m = 1 - \frac{0.10}{9.09} \approx 0.989.$$

The large-curvature direction (eigenvalue $M = 9.09$) is annihilated in a *single step* — its contraction factor is exactly 0. The small-curvature direction (eigenvalue $m = 0.10$) has contraction factor ≈ 0.989 , extremely close to 1, so error along that direction decays only very slowly, requiring many iterations. **The steep (large-eigenvalue) direction converges far faster than the flat (small-eigenvalue) direction** — this asymmetry is a direct consequence of ill-conditioning: a single step size cannot be simultaneously “just right” for both a steep and a flat direction.

$\eta = 2.1/M \approx 0.231$.

$$1 - \eta M = 1 - 2.1 = -1.1, \quad |-1.1| = 1.1 > 1,$$

so the error component along the M -eigendirection is *amplified* by a factor of 1.1 every iteration and grows without bound, even though the m -direction still contracts ($1 - \eta m \approx 0.977$, $|\cdot| < 1$). Since the initial error is assumed to have a nonzero component along the M -eigendirection, that component diverges geometrically, and **the overall iteration diverges** — this step size lies outside the interval $(0, 2/M)$ found above, confirming that $2/M$ is a hard stability threshold, not just a convenient bound.

Part c

Increasing λ does improve the condition number of $\mathbf{A}_\lambda$ (as shown in part a) and hence speeds up gradient descent — but the role of λ in ridge regression is to control the **bias–variance tradeoff of the statistical estimator**, not to make an optimization algorithm converge quickly. These are two entirely different objectives:

- **Optimization speed** measures how fast an iterative algorithm reaches *whatever* $\hat{\beta}_\lambda$ that particular λ defines. It is a purely computational property, governed by the condition number of $\mathbf{A}_\lambda$, and can be improved simply by making λ larger and larger.
- **Statistical quality** measures how well $\hat{\beta}_\lambda$ predicts on new data, which depends on the tradeoff between the bias introduced by shrinkage and the variance it removes. This is optimized at some finite, data-dependent λ^* , typically chosen by cross-validation or a held-out test set, as explored in Question 2.

If λ were chosen purely to minimize condition number / maximize convergence speed, the “optimal” choice would push $\lambda \rightarrow \infty$: the condition number $\kappa(\mathbf{A}_\lambda) = (\mu_{\max} + \lambda) / (\mu_{\min} + \lambda) \rightarrow 1$ as $\lambda \rightarrow \infty$, giving arbitrarily fast (in fact single-step) convergence. But this drives $\hat{\beta}_\lambda \rightarrow \mathbf{0}$, producing severe underfitting and enormous bias — a terrible predictive model, even though it would be “solved” instantly. Conflating the two criteria would sacrifice all statistical accuracy for computational convenience.

More fundamentally, for ridge regression there is rarely a need to make this tradeoff at all: $\hat{\beta}_\lambda$ has a **closed-form solution** $(\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p)^{-1} \mathbf{X}^\top \tilde{\mathbf{y}}$, so gradient descent (and its convergence speed) is not even necessary in practice — λ should be selected by whatever criterion best estimates out-of-sample prediction error, entirely independent of how any particular optimization algorithm happens to behave for that value of λ .

Question 5: Selecting the ridge penalty for real-estate prediction

Answer

Part a

```
import numpy as np
import pandas as pd

prices = pd.read_csv("data/realestate.csv")
folds = pd.read_csv("data/realestate-split-folds.csv")
df = prices.merge(folds, on="row_id")

predictors = ["date", "age", "distance", "stores", "latitude", "longitude"]

train = df[df["split"] == "train"].reset_index(drop=True)
test = df[df["split"] == "test"].reset_index(drop=True)

Xtrain_full = train[predictors].values
ytrain_full = train["price"].values
folds_arr = train["cv_fold"].values.astype(int)

Xtest = test[predictors].values
ytest = test["price"].values

n_train_full = Xtrain_full.shape[0]
p = Xtrain_full.shape[1]

# penalty grid
lambdas = np.concatenate([[0.0], 10 ** (-4 + 0.05 * np.arange(0, 121))])

def standardize(X):
    """center/scale using only the observations passed in; variance divided by n."""
    mean = X.mean(axis=0)
    Xc = X - mean
    var = (Xc ** 2).mean(axis=0) # divide by n, as specified
    scale = np.sqrt(var)
    return Xc / scale, mean, scale

def apply_scale(Xraw, mean, scale):
    return (Xraw - mean) / scale

def ridge_fit(Xs, ytilde, lam, n):
    pp = Xs.shape[1]
    A = Xs.T @ Xs + n * lam * np.eye(pp)
    return np.linalg.solve(A, Xs.T @ ytilde)
```

For each held-out fold, predictor means/scales are estimated **only** from the other nine folds, the model is fit on those nine folds, and validation MSE is computed on the held-out fold.

```

n_lam = len(lambdas)
fold_ids = np.unique(folds_arr)
K = len(fold_ids)
cv_errors = np.zeros((K, n_lam))

for ki, k in enumerate(fold_ids):
    val_mask = folds_arr == k
    tr_mask = ~val_mask

    Xtr_raw, ytr = Xtrain_full[tr_mask], ytrain_full[tr_mask]
    Xval_raw, yval = Xtrain_full[val_mask], ytrain_full[val_mask]
    n_tr = Xtr_raw.shape[0]

    Xtr_s, mean_k, scale_k = standardize(Xtr_raw)
    Xval_s = apply_scale(Xval_raw, mean_k, scale_k)

    ybar = ytr.mean()
    ytilde = ytr - ybar

    for li, lam in enumerate(lambdas):
        b = ridge_fit(Xtr_s, ytilde, lam, n_tr)
        pred_val = ybar + Xval_s @ b
        cv_errors[ki, li] = np.mean((yval - pred_val) ** 2)

mean_mse = cv_errors.mean(axis=0)
se_mse = cv_errors.std(axis=0, ddof=1) / np.sqrt(K)

idx_min = np.argmin(mean_mse)
lambda_min = lambdas[idx_min]
threshold = mean_mse[idx_min] + se_mse[idx_min]

eligible = np.where(mean_mse <= threshold)[0]
idx_1se = eligible[np.argmax(lambdas[eligible])]
lambda_1se = lambdas[idx_1se]

print(f"lambda_min = {lambda_min:.5f}, CV MSE = {mean_mse[idx_min]:.4f}, SE = {se_mse[idx_min]:.4f}")
print(f"threshold = {threshold:.4f}")
print(f"lambda_1se = {lambda_1se:.5f}, CV MSE = {mean_mse[idx_1se]:.4f}")

```

```

lambda_min = 0.01995, CV MSE = 82.9776, SE = 19.2299
threshold = 102.2076
lambda_1se = 1.58489, CV MSE = 101.5955

```

Results: $\lambda_{\min} \approx 0.0200$ (mean CV MSE ≈ 82.98 , SE ≈ 19.23), and applying the one-standard-error rule, $\lambda_{1se} \approx 1.5849$ (mean CV MSE $\approx 101.60 \leq 82.98 + 19.23 = 102.21$) — the largest λ in the grid whose mean CV error still falls within one SE of the minimum.

```

import matplotlib.pyplot as plt

lam_plot = lambdas.copy()
lam_plot[0] = 1e-4 / 2 # placeholder for lambda=0 so it can appear on a log axis

fig, ax = plt.subplots(figsize=(5.5, 3.3))
ax.errorbar(lam_plot, mean_mse, yerr=se_mse, fmt='o-', ms=3, lw=1,
            capsize=2, color='steelblue', label='CV mean MSE ± SE')
ax.axvline(lam_plot[idx_min], color='red', ls='--', label=f'$\lambda_{\min}={lambda_min:.4f}')
ax.axvline(lambda_1se, color='green', ls='--', label=f'$\lambda_{1se}={lambda_1se:.4f}')
ax.set_xscale('log')
ax.set_xlabel('$\lambda$ (log scale)')
ax.set_ylabel('Cross-validated MSE')
ax.set_title('10-fold CV: mean validation MSE vs. $\lambda$')
ax.legend()
fig.tight_layout()
plt.show()

```

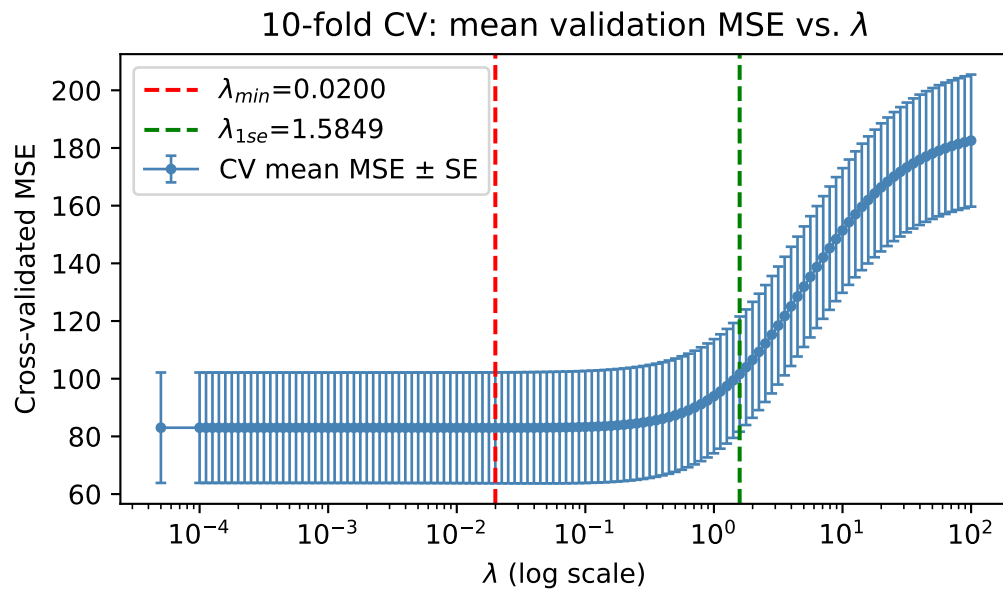


Figure 1: 10-fold CV: mean validation MSE vs. lambda

Part b

Means/scales are estimated **once**, from all 332 training observations, and held fixed.

```
Xs_all, mean_all, scale_all = standardize(Xtrain_full)
ybar_all = ytrain_full.mean()
ytilde_all = ytrain_full - ybar_all
n = n_train_full

U, S, Vt = np.linalg.svd(Xs_all, full_matrices=False) # singular values of Xs_all

gcv_vals = np.zeros(n_lam)
train_mse_vals = np.zeros(n_lam)
df_eff_vals = np.zeros(n_lam)

for li, lam in enumerate(lambdas):
    b = ridge_fit(Xs_all, ytilde_all, lam, n)
    fitted = ybar_all + Xs_all @ b
    mse_train = np.mean((ytrain_full - fitted) ** 2)
    df_eff = 1 + np.sum(S**2 / (S**2 + n * lam)) # includes unpenalized intercept
    gcv = mse_train / (1 - df_eff / n) ** 2

    train_mse_vals[li] = mse_train
    df_eff_vals[li] = df_eff
    gcv_vals[li] = gcv

idx_gcv = np.argmin(gcv_vals)
lambda_gcv = lambdas[idx_gcv]
print(f"lambda_GCV = {lambda_gcv:.5f}, GCV = {gcv_vals[idx_gcv]:.4f}, df_eff = {df_eff_vals[idx_gcv]:.4f}")
```

```
lambda_GCV = 0.02818, GCV = 83.5019, df_eff = 6.6887
```

```
fig2, ax2 = plt.subplots(figsize=(5.5, 3.3))
ax2.plot(lam_plot, gcv_vals, 'o-', ms=3, lw=1, color='darkorange', label='GCV')
ax2.axvline(lambda_gcv, color='purple', ls='--', label=f"$\\lambda_{{GCV}}$={lambda_gcv:.4f}")
ax2.set_xscale('log')
ax2.set_xlabel('$\\lambda$ (log scale)')
ax2.set_ylabel('GCV criterion')
ax2.set_title('GCV vs. $\\lambda$ (full training set)')
ax2.legend()
fig2.tight_layout()
plt.show()
```

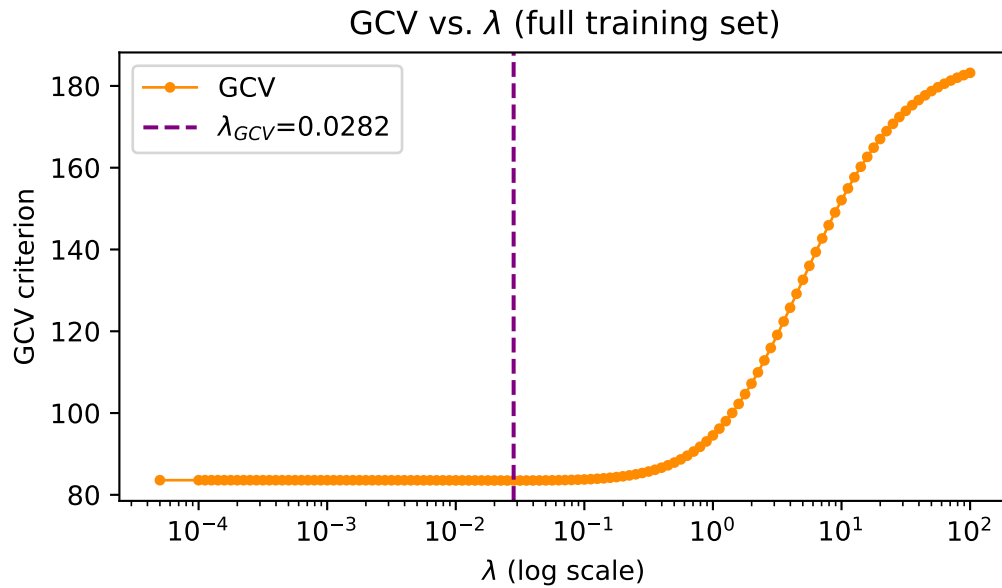


Figure 2: GCV criterion vs. lambda (full training set)

Result: $\lambda_{\text{GCV}} \approx 0.0282$, very close to $\lambda_{\min}$ from part (a) — as expected, since GCV approximates leave-one-out CV, and the GCV curve tracks the 10-fold CV curve closely across the whole grid.

Part c

All three λ choices — $\lambda_{\min}$, λ_{1se} , λ_{GCV} — plus OLS ($\lambda = 0$) are now fixed. Each is refit on **all 332 training observations**, and only now are the 82 test responses used.

```
chosen = {
    "OLS (lambda=0)": 0.0,
    "lambda_min":     lambda_min,
    "lambda_1se":     lambda_1se,
    "lambda_GCV":     lambda_gcv,
}

Xtest_s = apply_scale(Xtest, mean_all, scale_all)

rows = []
for name, lam in chosen.items():
    b = ridge_fit(Xs_all, ytilde_all, lam, n)
    fitted_train = ybar_all + Xs_all @ b
    mse_train = np.mean((ytrain_full - fitted_train) ** 2)

    fitted_test = ybar_all + Xtest_s @ b
    mse_test = np.mean((ytest - fitted_test) ** 2)

    df_eff = 1 + np.sum(S**2 / (S**2 + n * lam)) if lam > 0 else p + 1
    rows.append([name, lam, mse_train, mse_test, df_eff])

results_c = pd.DataFrame(rows, columns=["Model", "lambda", "Train MSE", "Test MSE", "df_eff"])
print(results_c.to_string(index=False, float_format=lambda x: f"{x:.4f}"))
```

	Model	lambda	Train MSE	Test MSE	df_eff
OLS	(lambda=0)	0.0000	80.0939	66.3225	7.0000
	lambda_min	0.0200	80.1354	65.9996	6.7732
	lambda_1se	1.5849	100.3736	81.1998	3.0329
	lambda_GCV	0.0282	80.1713	65.9028	6.6887

Model	λ	Train MSE	Test MSE	df_{eff}
OLS ($\lambda = 0$)	0	80.09	66.32	7.00
$\lambda_{\min}$	0.0200	80.14	66.00	6.77
λ_{1se}	1.5849	100.37	81.20	3.03
λ_{GCV}	0.0282	80.17	65.90	6.69

Training vs. test MSE, and effective df. Training MSE grows monotonically with λ ($80.09 \rightarrow 80.14 \rightarrow 80.17 \rightarrow 100.37$, following the increasing order $0 < \lambda_{\min} < \lambda_{\text{GCV}} < \lambda_{1\text{se}}$), because ridge always fits the training data slightly worse as the penalty grows. Effective degrees of freedom drop from the full $p + 1 = 7$ at OLS down to about 6.7–6.8 at $\lambda_{\min}/\lambda_{\text{GCV}}$ (very light shrinkage) and down to about 3.0 at $\lambda_{1\text{se}}$ (substantial shrinkage, roughly halving the effective model complexity).

$\lambda_{\min}$ and λ_{GCV} **nearly agree.** Both select very small penalties (0.020 and 0.028), essentially minor perturbations of OLS, and both slightly *improve* on OLS’s test MSE (66.00 and 65.90 vs. 66.32) — consistent with there being a small amount of collinearity/noise to damp out, but not much, since these six predictors (date, age, distance, stores, latitude, longitude) are not nearly as collinear as the engineered example in Question 2.

$\lambda_{1\text{se}}$ **chooses much more shrinkage** and, in this instance, ends up with a *worse* test MSE (81.20) than either OLS or the more lightly-penalized ridge fits. This is expected behavior for the one-SE rule: it deliberately trades a modest, statistically-indistinguishable increase in *estimated* CV error for a simpler, more parsimonious, lower-variance model — the CV curve shows that its mean MSE (101.60) is within one SE of the minimum, so the rule cannot statistically distinguish it from $\lambda_{\min}$ using the training-fold information alone. It is not guaranteed to win on any one particular test set draw.

Why you must not select the model with the smallest observed test MSE. In this table, λ_{GCV} happens to have the single lowest test MSE (65.90), even lower than $\lambda_{\min}$. But this is only true for this particular test split of 82 observations — the differences among OLS, $\lambda_{\min}$, and λ_{GCV} (66.32, 66.00, 65.90) are tiny relative to the sampling noise inherent in evaluating on 82 points (compare to the CV standard errors of ≈ 19 seen in part a). If we chose whichever model minimizes test MSE *after* looking at the test set, we would be tuning λ using the test data itself, defeating its purpose as an honest, independent estimate of generalization error — the resulting selected model’s test MSE would then be optimistically biased (a form of data leakage / “peeking”), and it would no longer provide a valid estimate of how the chosen model performs on genuinely new data. All three λ choices (and the decision rules that produced them) must be — and were — fixed using only the training folds and GCV before the test responses were ever examined; the test MSE table’s role is solely to *report and compare* the already-fixed choices, not to select among them.

Reference

The real-estate data come from I-Cheng Yeh’s [Real Estate Valuation dataset](#) in the UCI Machine Learning Repository. The distributed data retain the original observations and add only the course train-test and fold assignments.

explain-to-me skill

```
---
name: explain-to-me
description: Explain what a homework question is asking, without answering it. Use only when I explicitly ask to use explain-to-me.
metadata:
  version: "1.0.0"
---

# explain-to-me

Invoke this skill only when I explicitly ask you to use `explain-to-me` on a named question (e.g. "use explain-to-me on Question 2").

For that question, in plain language:

1. State the goal: what quantity, relationship, or property the question wants me to find or demonstrate.
2. State the required work: which steps, derivations, simulations, or plots I need to produce to answer it - as a checklist.
3. Point me to the relevant section of the Week 3 lectures if one applies:
  - [Ridge Regression: Stability Through Shrinkage] (https://teazrq.github.io/stat432rpy/topics/ridge-regression/ridge-regression-stability-through-shrinkage/)
  - [From a Penalized Objective to a Fitted Ridge Model] (https://teazrq.github.io/stat432rpy/topics/ridge-regression/optimizing-ridge-regression/)

Rules:

- Never compute, derive, simulate, or write code toward the actual answer - no numbers, formulas, or conclusions that the question asks for.
- Never open or reference `solutions-03.qmd` or any solutions file.
- Keep the explanation short: a goal statement, a checklist, and (if relevant) one lecture link. Do not restate the entire question.
```